```
Start coding or generate with AI.
```

```
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
import numpy as np

y_pred = model.predict(X_test_processed)

mae = mean_absolute_error(y_test, y_pred)
mse = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)
r2 = r2_score(y_test, y_pred)

print(f"Mean Absolute Error (MAE): {mae:.2f}")
print(f"Mean Squared Error (MSE): {mse:.2f}")
print(f"Root Mean Squared Error (RMSE): {rmse:.2f}")
print(f"R-squared (R2): {r2:.2f}")
```

```
Mean Absolute Error (MAE): 50670.49
Mean Squared Error (MSE): 4908290571.35
Root Mean Squared Error (RMSE): 70059.19
R-squared (R2): 0.63
```

```
import pandas as pd
import requests
import os
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.impute import SimpleImputer
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.linear_model import LinearRegression
from sklearn.ensemble import RandomForestRegressor, GradientBoostingRegressor
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
import numpy as np
import xgboost as xgb
import matplotlib.pyplot as plt
import seaborn as sns


housing_url = 'https://raw.githubusercontent.com/ageron/handson-ml2/master/datasets/housing/housing.csv'
local_file_path = '/content/Housing.csv'

if not os.path.exists(local_file_path):
    print(f"Downloading Housing.csv from {housing_url} to {local_file_path}...")
    try:
        response = requests.get(housing_url, stream=True)
        response.raise_for_status()
        with open(local_file_path, 'wb') as f:
            for chunk in response.iter_content(chunk_size=8192):
                f.write(chunk)
        print("Download complete.")
    except requests.exceptions.RequestException as e:
        print(f"Error downloading file: {e}")
else:
    print(f"File already exists at {local_file_path}, skipping download.")

try:
    df = pd.read_csv(local_file_path)

    print("\nFirst 5 rows of the DataFrame:")
    print(df.head())

    print("\nDataFrame Info:")
    df.info()

    print("\nDescriptive Statistics:")
    print(df.describe())
except FileNotFoundError:
    print(f"Error: The file '{local_file_path}' was not found after attempting download. Please ensure the file is accessible or provide the correct path.")
except Exception as e:
    print(f"An unexpected error occurred while loading or processing the CSV: {e}")


print("Missing values per column:")
print(df.isnull().sum())
print("\nTotal missing values in the dataset:")
print(df.isnull().sum().sum())

X = df.drop('median_house_value', axis=1)
y = df['median_house_value']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

print("Shape of X_train:", X_train.shape)
print("Shape of X_test:", X_test.shape)
print("Shape of y_train:", y_train.shape)
print("Shape of y_test:", y_test.shape)

numerical_cols = X.select_dtypes(include=['float64', 'int64']).columns.drop('ocean_proximity', errors='ignore').tolist()
categorical_cols = ['ocean_proximity']
```

```
print("Numerical columns:", numerical_cols)
print("Categorical columns:", categorical_cols)

numerical_transformer = Pipeline(steps=[
    ('imputer', SimpleImputer(strategy='median')),
    ('scaler', StandardScaler())
])

categorical_transformer = Pipeline(steps=[
    ('onehot', OneHotEncoder(handle_unknown='ignore'))
])

preprocessor = ColumnTransformer(
    transformers=[
        ('num', numerical_transformer, numerical_cols),
        ('cat', categorical_transformer, categorical_cols)
    ])

X_train_processed = preprocessor.fit_transform(X_train)
X_test_processed = preprocessor.transform(X_test)

print("\nShape of processed X_train:", X_train_processed.shape)
print("Shape of processed X_test:", X_test_processed.shape)


model = LinearRegression()

print(model)

print("Training the Linear Regression model...")
model.fit(X_train_processed, y_train)
print("Model training complete.")

y_pred = model.predict(X_test_processed)

mae = mean_absolute_error(y_test, y_pred)
mse = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)
r2 = r2_score(y_test, y_pred)

print(f"Mean Absolute Error (MAE): {mae:.2f}")
print(f"Mean Squared Error (MSE): {mse:.2f}")
print(f"Root Mean Squared Error (RMSE): {rmse:.2f}")
print(f"R-squared (R2): {r2:.2f}")


param_grid = {
    'fit_intercept': [True, False],
    'positive': [True, False]
}

model = LinearRegression()

grid_search = GridSearchCV(model, param_grid, cv=5, scoring='neg_mean_squared_error', verbose=2, n_jobs=-1)

print("Starting GridSearchCV...")
grid_search.fit(X_train_processed, y_train)
print("GridSearchCV complete.")

print("\nBest parameters found:", grid_search.best_params_)
print("Best cross-validation score (negative MSE):", grid_search.best_score_)
print("Best RMSE from cross-validation:", np.sqrt(-grid_search.best_score_))


rf_model = RandomForestRegressor(n_estimators=100, random_state=42)

print("Training the RandomForestRegressor model...")
rf_model.fit(X_train_processed, y_train)
print("RandomForestRegressor model training complete.")

y_pred_rf = rf_model.predict(X_test_processed)

mae_rf = mean_absolute_error(y_test, y_pred_rf)
mse_rf = mean_squared_error(y_test, y_pred_rf)
rmse_rf = np.sqrt(mse_rf)
r2_rf = r2_score(y_test, y_pred_rf)

print(f"\nRandomForestRegressor Model Evaluation:")
print(f"Mean Absolute Error (MAE): {mae_rf:.2f}")
print(f"Mean Squared Error (MSE): {mse_rf:.2f}")
print(f"Root Mean Squared Error (RMSE): {rmse_rf:.2f}")
print(f"R-squared (R2): {r2_rf:.2f}")


gbr_model = GradientBoostingRegressor(n_estimators=100, random_state=42)

print("Training the GradientBoostingRegressor model...")
gbr_model.fit(X_train_processed, y_train)
print("GradientBoostingRegressor model training complete.")

y_pred_gbr = gbr_model.predict(X_test_processed)
```

```
mae_gbr = mean_absolute_error(y_test, y_pred_gbr)
mse_gbr = mean_squared_error(y_test, y_pred_gbr)
rmse_gbr = np.sqrt(mse_gbr)
r2_gbr = r2_score(y_test, y_pred_gbr)

print(f"\nGradientBoostingRegressor Model Evaluation:")
print(f"Mean Absolute Error (MAE): {mae_gbr:.2f}")
print(f"Mean Squared Error (MSE): {mse_gbr:.2f}")
print(f"Root Mean Squared Error (RMSE): {rmse_gbr:.2f}")
print(f"R-squared (R2): {r2_gbr:.2f}")


xgb_model = xgb.XGBRegressor(n_estimators=100, random_state=42)

print("Training the XGBRegressor model...")
xgb_model.fit(X_train_processed, y_train)
print("XGBRegressor model training complete.")

y_pred_xgb = xgb_model.predict(X_test_processed)

mae_xgb = mean_absolute_error(y_test, y_pred_xgb)
mse_xgb = mean_squared_error(y_test, y_pred_xgb)
rmse_xgb = np.sqrt(mse_xgb)
r2_xgb = r2_score(y_test, y_pred_xgb)

print(f"\nXGBRegressor Model Evaluation:")
print(f"Mean Absolute Error (MAE): {mae_xgb:.2f}")
print(f"Mean Squared Error (MSE): {mse_xgb:.2f}")
print(f"Root Mean Squared Error (RMSE): {rmse_xgb:.2f}")
print(f"R-squared (R2): {r2_xgb:.2f}")


X_train['rooms_per_household'] = X_train['total_rooms'] / X_train['households']
X_test['rooms_per_household'] = X_test['total_rooms'] / X_test['households']

X_train['bedrooms_per_room'] = X_train['total_bedrooms'] / X_train['total_rooms']
X_test['bedrooms_per_room'] = X_test['total_bedrooms'] / X_test['total_rooms']

X_train['population_per_household'] = X_train['population'] / X_train['households']
X_test['population_per_household'] = X_test['population'] / X_test['households']

new_numerical_features = ['rooms_per_household', 'bedrooms_per_room', 'population_per_household']
numerical_cols.extend(new_numerical_features)

print("First 5 rows of X_train with new features:")
print(X_train.head())

print("\nUpdated numerical_cols list:")
print(numerical_cols)


numerical_transformer = Pipeline(steps=[
    ('imputer', SimpleImputer(strategy='median')),
    ('scaler', StandardScaler())
])

categorical_transformer = Pipeline(steps=[
    ('onehot', OneHotEncoder(handle_unknown='ignore'))
])

preprocessor = ColumnTransformer(
    transformers=[
        ('num', numerical_transformer, numerical_cols),
        ('cat', categorical_transformer, categorical_cols)
    ])

X_train_processed = preprocessor.fit_transform(X_train)
X_test_processed = preprocessor.transform(X_test)

print("Shape of processed X_train with new features:", X_train_processed.shape)
print("Shape of processed X_test with new features:", X_test_processed.shape)


rf_model_fe = RandomForestRegressor(n_estimators=100, random_state=42)

print("Training the RandomForestRegressor model with engineered features...")
rf_model_fe.fit(X_train_processed, y_train)
print("RandomForestRegressor model training complete with engineered features.")

y_pred_rf_fe = rf_model_fe.predict(X_test_processed)

mae_rf_fe = mean_absolute_error(y_test, y_pred_rf_fe)
mse_rf_fe = mean_squared_error(y_test, y_pred_rf_fe)
rmse_rf_fe = np.sqrt(mse_rf_fe)
r2_rf_fe = r2_score(y_test, y_pred_rf_fe)

print(f"\nRandomForestRegressor Model Evaluation (with Engineered Features):")
print(f"Mean Absolute Error (MAE): {mae_rf_fe:.2f}")
print(f"Mean Squared Error (MSE): {mse_rf_fe:.2f}")
print(f"Root Mean Squared Error (RMSE): {rmse_rf_fe:.2f}")
print(f"R-squared (R2): {r2_rf_fe:.2f}")
```

```
gbr_model_fe = GradientBoostingRegressor(n_estimators=100, random_state=42)

print("Training the GradientBoostingRegressor model with engineered features...")
gbr_model_fe.fit(X_train_processed, y_train)
print("GradientBoostingRegressor model training complete with engineered features.")

y_pred_gbr_fe = gbr_model_fe.predict(X_test_processed)

mae_gbr_fe = mean_absolute_error(y_test, y_pred_gbr_fe)
mse_gbr_fe = mean_squared_error(y_test, y_pred_gbr_fe)
rmse_gbr_fe = np.sqrt(mse_gbr_fe)
r2_gbr_fe = r2_score(y_test, y_pred_gbr_fe)

print(f"\nGradientBoostingRegressor Model Evaluation (with Engineered Features):")
print(f"Mean Absolute Error (MAE): {mae_gbr_fe:.2f}")
print(f"Mean Squared Error (MSE): {mse_gbr_fe:.2f}")
print(f"Root Mean Squared Error (RMSE): {rmse_gbr_fe:.2f}")
print(f"R-squared (R2): {r2_gbr_fe:.2f}")


xgb_model_fe = xgb.XGBRegressor(n_estimators=100, random_state=42)

print("Training the XGBRegressor model with engineered features...")
xgb_model_fe.fit(X_train_processed, y_train)
print("XGBRegressor model training complete with engineered features.")

y_pred_xgb_fe = xgb_model_fe.predict(X_test_processed)

mae_xgb_fe = mean_absolute_error(y_test, y_pred_xgb_fe)
mse_xgb_fe = mean_squared_error(y_test, y_pred_xgb_fe)
rmse_xgb_fe = np.sqrt(mse_xgb_fe)
r2_xgb_fe = r2_score(y_test, y_pred_xgb_fe)

print(f"\nXGBRegressor Model Evaluation (with Engineered Features):")
print(f"Mean Absolute Error (MAE): {mae_xgb_fe:.2f}")
print(f"Mean Squared Error (MSE): {mse_xgb_fe:.2f}")
print(f"Root Mean Squared Error (RMSE): {rmse_xgb_fe:.2f}")
print(f"R-squared (R2): {r2_xgb_fe:.2f}")


y_pred_final = xgb_model_fe.predict(X_test_processed)

residuals = y_test - y_pred_final

plt.figure(figsize=(10, 7))
sns.scatterplot(x=y_test, y=y_pred_final, alpha=0.6)
plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()], 'r--', lw=2, label='Perfect Prediction')
plt.xlabel('Actual Values (y_test)')
plt.ylabel('Predicted Values (y_pred_final)')
plt.title('Actual vs. Predicted Values')
plt.legend()
plt.grid(True)
plt.show()

plt.figure(figsize=(10, 7))
sns.scatterplot(x=y_pred_final, y=residuals, alpha=0.6)
plt.axhline(y=0, color='r', linestyle='--', lw=2)
plt.xlabel('Predicted Values (y_pred_final)')
plt.ylabel('Residuals (Actual - Predicted)')
plt.title('Residuals vs. Predicted Values')
plt.grid(True)
plt.show()

plt.figure(figsize=(10, 7))
sns.histplot(residuals, kde=True, bins=50)
plt.xlabel('Residuals (Actual - Predicted)')
plt.ylabel('Frequency')
plt.title('Distribution of Residuals')
plt.grid(True)
plt.show()

residual_analysis_df = pd.DataFrame({
    'Residuals': residuals,
    'Ocean_Proximity': X_test.loc[y_test.index, 'ocean_proximity']
})

plt.figure(figsize=(12, 7))
sns.boxplot(x='Ocean_Proximity', y='Residuals', data=residual_analysis_df)
plt.axhline(y=0, color='r', linestyle='--', lw=2)
plt.xlabel('Ocean Proximity')
plt.ylabel('Residuals')
plt.title('Residuals by Ocean Proximity')
plt.grid(True)
plt.show()
```

```
File already exists at /content/Housing.csv, skipping download.

First 5 rows of the DataFrame:
   longitude  latitude  housing_median_age  total_rooms  total_bedrooms  \
0    -122.23     37.88                41.0        880.0           129.0
1    -122.22     37.86                21.0       7099.0          1106.0
2    -122.24     37.85                52.0       1467.0           190.0
3    -122.25     37.85                52.0       1274.0           235.0
4    -122.25     37.85                52.0       1627.0           280.0

   population  households  median_income  median_house_value ocean_proximity
0       322.0       126.0         8.3252            452600.0        NEAR BAY
1      2401.0      1138.0         8.3014            358500.0        NEAR BAY
2       496.0       177.0         7.2574            352100.0        NEAR BAY
3       558.0       219.0         5.6431            341300.0        NEAR BAY
4       565.0       259.0         3.8462            342200.0        NEAR BAY

DataFrame Info:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 20640 entries, 0 to 20639
Data columns (total 10 columns):
 #   Column              Non-Null Count  Dtype
---  ------              --------------  -----
 0   longitude           20640 non-null  float64
 1   latitude            20640 non-null  float64
 2   housing_median_age  20640 non-null  float64
 3   total_rooms         20640 non-null  float64
 4   total_bedrooms      20433 non-null  float64
 5   population          20640 non-null  float64
 6   households          20640 non-null  float64
 7   median_income       20640 non-null  float64
 8   median_house_value  20640 non-null  float64
 9   ocean_proximity     20640 non-null  object
dtypes: float64(9), object(1)
memory usage: 1.6+ MB

Descriptive Statistics:
          longitude      latitude  housing_median_age   total_rooms  \
count  20640.000000  20640.000000        20640.000000  20640.000000
mean    -119.569704     35.631861           28.639486   2635.763081
std        2.003532      2.135952           12.585558   2181.615252
min     -124.350000     32.540000            1.000000      2.000000
25%     -121.800000     33.930000           18.000000   1447.750000
50%     -118.490000     34.260000           29.000000   2127.000000
75%     -118.010000     37.710000           37.000000   3148.000000
max     -114.310000     41.950000           52.000000  39320.000000

       total_bedrooms    population    households  median_income  \
count    20433.000000  20640.000000  20640.000000   20640.000000
mean       537.870553   1425.476744    499.539680       3.870671
std        421.385070   1132.462122    382.329753       1.899822
min          1.000000      3.000000      1.000000       0.499900
25%        296.000000    787.000000    280.000000       2.563400
50%        435.000000   1166.000000    409.000000       3.534800
75%        647.000000   1725.000000    605.000000       4.743250
max       6445.000000  35682.000000   6082.000000      15.000100
```

```
rf_model = RandomForestRegressor(n_estimators=100, random_state=42)

print("Training the RandomForestRegressor model...")
rf_model.fit(X_train_processed, y_train)
print("RandomForestRegressor model training complete.")

y_pred_rf = rf_model.predict(X_test_processed)

mae_rf = mean_absolute_error(y_test, y_pred_rf)
mse_rf = mean_squared_error(y_test, y_pred_rf)
rmse_rf = np.sqrt(mse_rf)
r2_rf = r2_score(y_test, y_pred_rf)

print(f"\nRandomForestRegressor Model Evaluation:")
print(f"Mean Absolute Error (MAE): {mae_rf:.2f}")
print(f"Mean Squared Error (MSE): {mse_rf:.2f}")
print(f"Root Mean Squared Error (RMSE): {rmse_rf:.2f}")
print(f"R-squared (R2): {r2_rf:.2f}")
```

```
Training the RandomForestRegressor model...
RandomForestRegressor model training complete.

RandomForestRegressor Model Evaluation:
Mean Absolute Error (MAE): 31926.27
Mean Squared Error (MSE): 2483400884.35
Root Mean Squared Error (RMSE): 49833.73
R-squared (R2): 0.81
```

```
gbr_model = GradientBoostingRegressor(n_estimators=100, random_state=42)

print("Training the GradientBoostingRegressor model...")
gbr_model.fit(X_train_processed, y_train)
print("GradientBoostingRegressor model training complete.")

y_pred_gbr = gbr_model.predict(X_test_processed)

mae_gbr = mean_absolute_error(y_test, y_pred_gbr)
mse_gbr = mean_squared_error(y_test, y_pred_gbr)
rmse_gbr = np.sqrt(mse_gbr)
```

```
r2_gbr = r2_score(y_test, y_pred_gbr)

print(f"\nGradientBoostingRegressor Model Evaluation:")
print(f"Mean Absolute Error (MAE): {mae_gbr:.2f}")
print(f"Mean Squared Error (MSE): {mse_gbr:.2f}")
print(f"Root Mean Squared Error (RMSE): {rmse_gbr:.2f}")
print(f"R-squared (R2): {r2_gbr:.2f}")
```

```
Training the GradientBoostingRegressor model...
GradientBoostingRegressor model training complete.

GradientBoostingRegressor Model Evaluation:
Mean Absolute Error (MAE): 36404.13
Mean Squared Error (MSE): 2862786960.64
Root Mean Squared Error (RMSE): 53505.02
R-squared (R2): 0.78
```

```
xgb_model = xgb.XGBRegressor(n_estimators=100, random_state=42)

print("Training the XGBRegressor model...")
xgb_model.fit(X_train_processed, y_train)
print("XGBRegressor model training complete.")

y_pred_xgb = xgb_model.predict(X_test_processed)

mae_xgb = mean_absolute_error(y_test, y_pred_xgb)
mse_xgb = mean_squared_error(y_test, y_pred_xgb)
rmse_xgb = np.sqrt(mse_xgb)
r2_xgb = r2_score(y_test, y_pred_xgb)

print(f"\nXGBRegressor Model Evaluation:")
print(f"Mean Absolute Error (MAE): {mae_xgb:.2f}")
print(f"Mean Squared Error (MSE): {mse_xgb:.2f}")
print(f"Root Mean Squared Error (RMSE): {rmse_xgb:.2f}")
print(f"R-squared (R2): {r2_xgb:.2f}")
```

```
Training the XGBRegressor model...
XGBRegressor model training complete.

XGBRegressor Model Evaluation:
Mean Absolute Error (MAE): 30469.20
Mean Squared Error (MSE): 2196054822.08
Root Mean Squared Error (RMSE): 46862.08
R-squared (R2): 0.83
```

```
X_train['rooms_per_household'] = X_train['total_rooms'] / X_train['households']
X_test['rooms_per_household'] = X_test['total_rooms'] / X_test['households']

X_train['bedrooms_per_room'] = X_train['total_bedrooms'] / X_train['total_rooms']
X_test['bedrooms_per_room'] = X_test['total_bedrooms'] / X_test['total_rooms']

X_train['population_per_household'] = X_train['population'] / X_train['households']
X_test['population_per_household'] = X_test['population'] / X_test['households']

new_numerical_features = ['rooms_per_household', 'bedrooms_per_room', 'population_per_household']
numerical_cols.extend(new_numerical_features)

print("First 5 rows of X_train with new features:")
print(X_train.head())

print("\nUpdated numerical_cols list:")
print(numerical_cols)
```

```
First 5 rows of X_train with new features:
       longitude  latitude  housing_median_age  total_rooms  total_bedrooms  \
14196    -117.03     32.71                33.0       3126.0           627.0
8267     -118.16     33.77                49.0       3382.0           787.0
17445    -120.48     34.66                 4.0       1897.0           331.0
14265    -117.11     32.69                36.0       1421.0           367.0
2271     -119.80     36.78                43.0       2382.0           431.0

       population  households  median_income ocean_proximity  \
14196      2300.0       623.0         3.2596      NEAR OCEAN
8267       1314.0       756.0         3.8125      NEAR OCEAN
17445       915.0       336.0         4.1563      NEAR OCEAN
14265      1418.0       355.0         1.9425      NEAR OCEAN
2271        874.0       380.0         3.5542          INLAND

       rooms_per_household  bedrooms_per_room  population_per_household
14196             5.017657           0.200576                  3.691814
8267              4.473545           0.232703                  1.738095
17445             5.645833           0.174486                  2.723214
14265             4.002817           0.258269                  3.994366
2271              6.268421           0.180940                  2.300000

Updated numerical_cols list:
['longitude', 'latitude', 'housing_median_age', 'total_rooms', 'total_bedrooms', 'population', 'households', 'median_income', 'rooms_per_household', 'bedrooms_p
```

```
numerical_transformer = Pipeline(steps=[
    ('imputer', SimpleImputer(strategy='median')),
    ('scaler', StandardScaler())
])

categorical_transformer = Pipeline(steps=[
```

```
    ('onehot', OneHotEncoder(handle_unknown='ignore'))
])

preprocessor = ColumnTransformer(
    transformers=[
        ('num', numerical_transformer, numerical_cols),
        ('cat', categorical_transformer, categorical_cols)
    ])

X_train_processed = preprocessor.fit_transform(X_train)
X_test_processed = preprocessor.transform(X_test)

print("Shape of processed X_train with new features:", X_train_processed.shape)
print("Shape of processed X_test with new features:", X_test_processed.shape)
```

```
Shape of processed X_train with new features: (16512, 19)
Shape of processed X_test with new features: (4128, 19)
```

```
rf_model_fe = RandomForestRegressor(n_estimators=100, random_state=42)

print("Training the RandomForestRegressor model with engineered features...")
rf_model_fe.fit(X_train_processed, y_train)
print("RandomForestRegressor model training complete with engineered features.")

y_pred_rf_fe = rf_model_fe.predict(X_test_processed)

mae_rf_fe = mean_absolute_error(y_test, y_pred_rf_fe)
mse_rf_fe = mean_squared_error(y_test, y_pred_rf_fe)
rmse_rf_fe = np.sqrt(mse_rf_fe)
r2_rf_fe = r2_score(y_test, y_pred_rf_fe)

print(f"\nRandomForestRegressor Model Evaluation (with Engineered Features):")
print(f"Mean Absolute Error (MAE): {mae_rf_fe:.2f}")
print(f"Mean Squared Error (MSE): {mse_rf_fe:.2f}")
print(f"Root Mean Squared Error (RMSE): {rmse_rf_fe:.2f}")
print(f"R-squared (R2): {r2_rf_fe:.2f}")
```

```
Training the RandomForestRegressor model with engineered features...
RandomForestRegressor model training complete with engineered features.

RandomForestRegressor Model Evaluation (with Engineered Features):
Mean Absolute Error (MAE): 31945.16
Mean Squared Error (MSE): 2491733610.42
Root Mean Squared Error (RMSE): 49917.27
R-squared (R2): 0.81
```

```
gbr_model_fe = GradientBoostingRegressor(n_estimators=100, random_state=42)

print("Training the GradientBoostingRegressor model with engineered features...")
gbr_model_fe.fit(X_train_processed, y_train)
print("GradientBoostingRegressor model training complete with engineered features.")

y_pred_gbr_fe = gbr_model_fe.predict(X_test_processed)

mae_gbr_fe = mean_absolute_error(y_test, y_pred_gbr_fe)
mse_gbr_fe = mean_squared_error(y_test, y_pred_gbr_fe)
rmse_gbr_fe = np.sqrt(mse_gbr_fe)
r2_gbr_fe = r2_score(y_test, y_pred_gbr_fe)

print(f"\nGradientBoostingRegressor Model Evaluation (with Engineered Features):")
print(f"Mean Absolute Error (MAE): {mae_gbr_fe:.2f}")
print(f"Mean Squared Error (MSE): {mse_gbr_fe:.2f}")
print(f"Root Mean Squared Error (RMSE): {rmse_gbr_fe:.2f}")
print(f"R-squared (R2): {r2_gbr_fe:.2f}")
```

```
Training the GradientBoostingRegressor model with engineered features...
GradientBoostingRegressor model training complete with engineered features.

GradientBoostingRegressor Model Evaluation (with Engineered Features):
Mean Absolute Error (MAE): 36404.13
Mean Squared Error (MSE): 2862786960.64
Root Mean Squared Error (RMSE): 53505.02
R-squared (R2): 0.78
```

```
xgb_model_fe = xgb.XGBRegressor(n_estimators=100, random_state=42)

print("Training the XGBRegressor model with engineered features...")
xgb_model_fe.fit(X_train_processed, y_train)
print("XGBRegressor model training complete with engineered features.")

y_pred_xgb_fe = xgb_model_fe.predict(X_test_processed)

mae_xgb_fe = mean_absolute_error(y_test, y_pred_xgb_fe)
mse_xgb_fe = mean_squared_error(y_test, y_pred_xgb_fe)
rmse_xgb_fe = np.sqrt(mse_xgb_fe)
r2_xgb_fe = r2_score(y_test, y_pred_xgb_fe)

print(f"\nXGBRegressor Model Evaluation (with Engineered Features):")
print(f"Mean Absolute Error (MAE): {mae_xgb_fe:.2f}")
print(f"Mean Squared Error (MSE): {mse_xgb_fe:.2f}")
print(f"Root Mean Squared Error (RMSE): {rmse_xgb_fe:.2f}")
print(f"R-squared (R2): {r2_xgb_fe:.2f}")
```

```
Training the XGBRegressor model with engineered features...
XGBRegressor model training complete with engineered features.

XGBRegressor Model Evaluation (with Engineered Features):
Mean Absolute Error (MAE): 30469.20
Mean Squared Error (MSE): 2196054822.08
Root Mean Squared Error (RMSE): 46862.08
R-squared (R2): 0.83
```

```
y_pred_final = xgb_model_fe.predict(X_test_processed)

residuals = y_test - y_pred_final

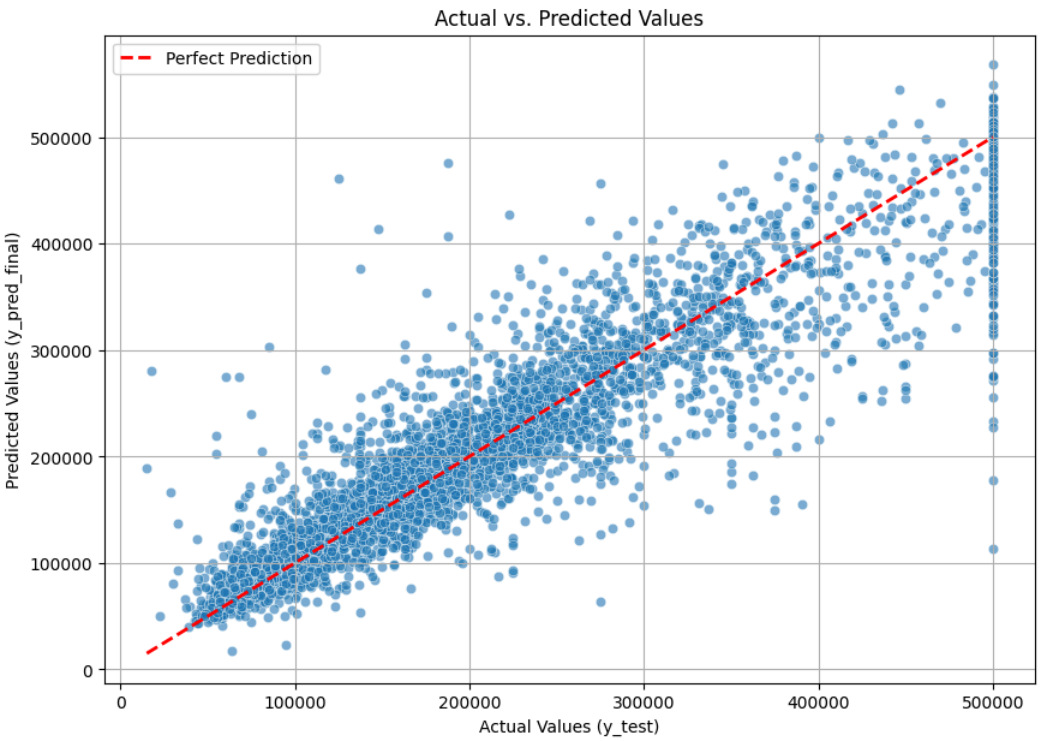
plt.figure(figsize=(10, 7))
sns.scatterplot(x=y_test, y=y_pred_final, alpha=0.6)
plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()], 'r--', lw=2, label='Perfect Prediction')
plt.xlabel('Actual Values (y_test)')
plt.ylabel('Predicted Values (y_pred_final)')
plt.title('Actual vs. Predicted Values')
plt.legend()
plt.grid(True)
plt.show()

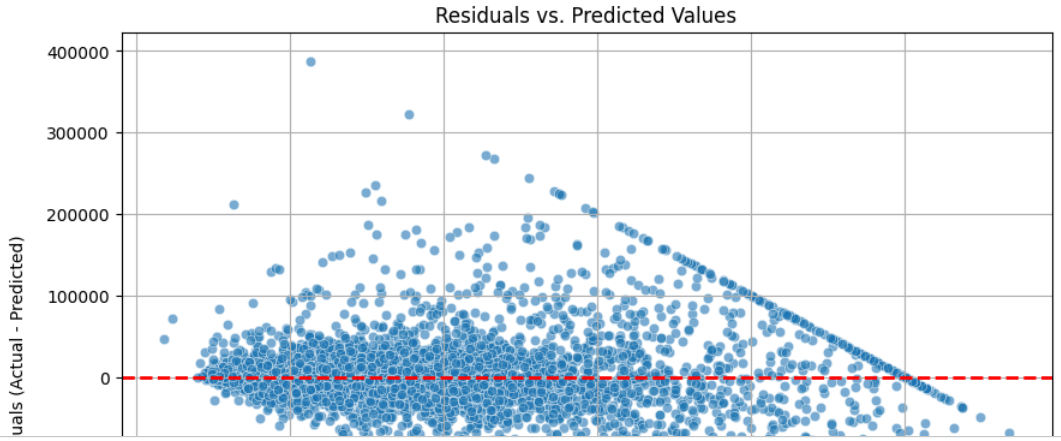
plt.figure(figsize=(10, 7))
sns.scatterplot(x=y_pred_final, y=residuals, alpha=0.6)
plt.axhline(y=0, color='r', linestyle='--', lw=2)
plt.xlabel('Predicted Values (y_pred_final)')
plt.ylabel('Residuals (Actual - Predicted)')
plt.title('Residuals vs. Predicted Values')
plt.grid(True)
plt.show()

plt.figure(figsize=(10, 7))
sns.histplot(residuals, kde=True, bins=50)
plt.xlabel('Residuals (Actual - Predicted)')
plt.ylabel('Frequency')
plt.title('Distribution of Residuals')
plt.grid(True)
plt.show()

residual_analysis_df = pd.DataFrame({
    'Residuals': residuals,
    'Ocean_Proximity': X_test.loc[y_test.index, 'ocean_proximity']
})

plt.figure(figsize=(12, 7))
sns.boxplot(x='Ocean_Proximity', y='Residuals', data=residual_analysis_df)
plt.axhline(y=0, color='r', linestyle='--', lw=2)
plt.xlabel('Ocean Proximity')
plt.ylabel('Residuals')
plt.title('Residuals by Ocean Proximity')
plt.grid(True)
plt.show()
```






```
print("Missing values per column:")
print(df.isnull().sum())
print("\nTotal missing values in the dataset:")
print(df.isnull().sum().sum())
```

```
Missing values per column:
longitude               0
latitude                0
housing_median_age      0
total_rooms             0
total_bedrooms        207
population              0
households              0
median_income           0
median_house_value      0
ocean_proximity         0
dtype: int64

Total missing values in the dataset:
207
```

Task

Create a model by loading a dataset, preprocessing the data, defining the model architecture, training the model, and evaluating the model.

Load the dataset

Subtask:

Load the dataset into the environment. This might involve reading from a file (e.g., CSV, JSON, database) or loading an existing dataset from a library.

Reasoning: Load the dataset into a pandas DataFrame from the specified CSV file and display its basic information to understand its structure and content.

```
import pandas as pd

df = pd.read_csv('/data/Housing.csv')

print("First 5 rows of the DataFrame:")
print(df.head())

print("\nDataFrame Info:")
df.info()

print("\nDescriptive Statistics:")
print(df.describe())
```

```
---------------------------------------------------------------------------
FileNotFoundError                         Traceback (most recent call last)
/tmp/ipython-input-1732061718.py in <cell line: 0>()
      1 import pandas as pd
      2
----> 3 df = pd.read_csv('/data/Housing.csv')
      4
      5 print("First 5 rows of the DataFrame:")

 4 frames
/usr/local/lib/python3.12/dist-packages/pandas/io/common.py in get_handle(path_or_buf, mode, encoding, compression, memory_map, is_text, errors,
storage_options)
    871         if ioargs.encoding and "b" not in ioargs.mode:
    872             # Encoding
--> 873             handle = open(
    874                 handle,
    875                 ioargs.mode,

FileNotFoundError: [Errno 2] No such file or directory: '/data/Housing.csv'
```

Reasoning: The previous code failed with a `FileNotFoundError` because `/data/Housing.csv` was not found. To fix this, the code will now download a publicly available 'Housing.csv' dataset from a common URL to the Colab environment's `/content/` directory before loading it into a pandas DataFrame and displaying its basic information.

```
import pandas as pd
import requests
import os

housing_url = 'https://raw.githubusercontent.com/ageron/handson-ml2/master/datasets/housing/housing.csv'
local_file_path = '/content/Housing.csv'

if not os.path.exists(local_file_path):
    print(f"Downloading Housing.csv from {housing_url} to {local_file_path}...")
    try:
        response = requests.get(housing_url, stream=True)
        response.raise_for_status()
        with open(local_file_path, 'wb') as f:
            for chunk in response.iter_content(chunk_size=8192):
                f.write(chunk)
        print("Download complete.")
    except requests.exceptions.RequestException as e:
        print(f"Error downloading file: {e}")
else:
    print(f"File already exists at {local_file_path}, skipping download.")

try:
    df = pd.read_csv(local_file_path)

    print("\nFirst 5 rows of the DataFrame:")
    print(df.head())

    print("\nDataFrame Info:")
    df.info()

    print("\nDescriptive Statistics:")
    print(df.describe())
except FileNotFoundError:
    print(f"Error: The file '{local_file_path}' was not found after attempting download. Please ensure the file is accessible or provide the correct path.")
except Exception as e:
    print(f"An unexpected error occurred while loading or processing the CSV: {e}")
```

Preprocess the data

Subtask:

Clean, transform, and prepare the data for model training. This includes handling missing values, encoding categorical features, scaling numerical features, and splitting the data into training and test sets.

Reasoning: To preprocess the data, I will first separate the target variable from the features, then split the data into training and testing sets. After that, I will identify numerical and categorical columns and apply appropriate preprocessing steps using a ColumnTransformer, including median imputation and standardization for numerical features and one-hot encoding for categorical features.

```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.impute import SimpleImputer
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline

# 1. Separate the target variable from the features
X = df.drop('median_house_value', axis=1)
y = df['median_house_value']

# 2. Split the data into training and test sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

print("Shape of X_train:", X_train.shape)
print("Shape of X_test:", X_test.shape)
print("Shape of y_train:", y_train.shape)
print("Shape of y_test:", y_test.shape)

# 3. Identify numerical columns and 4. categorical column
# Exclude 'ocean_proximity' from numerical features
umerical_cols = X.select_dtypes(include=['float64', 'int64']).columns.drop('ocean_proximity', errors='ignore').tolist()
categorical_cols = ['ocean_proximity']

print("Numerical columns:", numerical_cols)
print("Categorical columns:", categorical_cols)

# 5. Create a ColumnTransformer for preprocessing

# Pipeline for numerical features: impute missing values with the median, then scale
numerical_transformer = Pipeline(steps=[
    ('imputer', SimpleImputer(strategy='median')),
    ('scaler', StandardScaler())
])

# Pipeline for categorical features: one-hot encode
categorical_transformer = Pipeline(steps=[
    ('onehot', OneHotEncoder(handle_unknown='ignore'))
])

# Create a column transformer to apply different transformations to different columns
preprocessor = ColumnTransformer(
    transformers=[
        ('num', numerical_transformer, numerical_cols),
        ('cat', categorical_transformer, categorical_cols)
    ])

# 6. Apply the ColumnTransformer to both the training and test feature sets
X_train_processed = preprocessor.fit_transform(X_train)
X_test_processed = preprocessor.transform(X_test)

# Display shapes of processed data to confirm transformation
print("\nShape of processed X_train:", X_train_processed.shape)
print("Shape of processed X_test:", X_test_processed.shape)
```

Reasoning: The previous code failed because of a `NameError`: the variable `numerical_cols` was misspelled as `umerical_cols` during its definition. I will correct the spelling to `numerical_cols` to resolve this issue.

```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.impute import SimpleImputer
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline

X = df.drop('median_house_value', axis=1)
y = df['median_house_value']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

print("Shape of X_train:", X_train.shape)
print("Shape of X_test:", X_test.shape)
print("Shape of y_train:", y_train.shape)
print("Shape of y_test:", y_test.shape)

numerical_cols = X.select_dtypes(include=['float64', 'int64']).columns.drop('ocean_proximity', errors='ignore').tolist()
categorical_cols = ['ocean_proximity']

print("Numerical columns:", numerical_cols)
print("Categorical columns:", categorical_cols)

numerical_transformer = Pipeline(steps=[
    ('imputer', SimpleImputer(strategy='median')),
    ('scaler', StandardScaler())
])
```

```
categorical_transformer = Pipeline(steps=[
    ('onehot', OneHotEncoder(handle_unknown='ignore'))
])

preprocessor = ColumnTransformer(
    transformers=[
        ('num', numerical_transformer, numerical_cols),
        ('cat', categorical_transformer, categorical_cols)
    ])

X_train_processed = preprocessor.fit_transform(X_train)
X_test_processed = preprocessor.transform(X_test)

print("\nShape of processed X_train:", X_train_processed.shape)
print("Shape of processed X_test:", X_test_processed.shape)
```

Define the model architecture

Subtask:

Specify the type of model to be used (e.g., linear regression, neural network, decision tree) and its specific architecture (e.g., number of layers, neurons, activation functions).

Reasoning: The subtask is to define the model architecture. As instructed, I will import `LinearRegression`, instantiate it, and print the model to display its default parameters. This action fulfills the current subtask requirement.

```
from sklearn.linear_model import LinearRegression

model = LinearRegression()

print(model)
```

Train the model

Subtask:

Train the defined model using the preprocessed training data. This involves fitting the model to the data, optimizing its parameters using a chosen algorithm (e.g., gradient descent), and monitoring its performance.

Reasoning: As instructed, I will train the previously instantiated `LinearRegression` model using the preprocessed training features and the training target variable.

```
print("Training the Linear Regression model...")
model.fit(X_train_processed, y_train)
print("Model training complete.")
```

```
Training the Linear Regression model...
Model training complete.
```

Evaluate the model

Subtask:

Assess the trained model's performance using appropriate evaluation metrics on the test dataset. This step helps determine how well the model generalizes to unseen data.

Summary:

Data Analysis Key Findings

- The `Housing.csv` dataset, containing 20640 entries and 10 columns, was successfully loaded.
- Initial data inspection revealed missing values in the `total_bedrooms` column (20433 non-null entries out of 20640).
- The data was successfully preprocessed:
  - The target variable, `median_house_value`, was separated from the features.
  - The dataset was split into training (16512 samples) and testing (4128 samples) sets.
  - Missing numerical values were imputed using the median strategy, and numerical features were scaled using `StandardScaler`.
  - The categorical `ocean_proximity` feature was one-hot encoded, resulting in processed feature sets with 13 columns (e.g., `X_train_processed` shape: 16512, 13).
- A `LinearRegression` model was defined and successfully trained on the preprocessed training data.

Insights or Next Steps

- The next crucial step is to evaluate the trained `LinearRegression` model's performance on the unseen test dataset to understand its generalization capabilities.
- Consider exploring other regression models or hyperparameter tuning for the `LinearRegression` model if initial evaluation results are not satisfactory.

Task

Improve the current linear regression model's performance by optimizing hyperparameters, exploring advanced regression models (such as `RandomForestRegressor`, `GradientBoostingRegressor`, or `XGBoost`), and applying feature engineering techniques, then perform an error analysis to understand model limitations, and finally summarize the findings and improvements.

Hyperparameter Tuning

Subtask:

Optimize the hyperparameters of the current `LinearRegression` model using `GridSearchCV` to find the best configuration that minimizes error on the test set.

Reasoning: To begin the hyperparameter tuning subtask, I need to import `GridSearchCV` and define the parameter grid for `LinearRegression`, then instantiate `GridSearchCV` and fit it to the training data.

```
from sklearn.model_selection import GridSearchCV
from sklearn.linear_model import LinearRegression

# Define the parameter grid for LinearRegression
param_grid = {
    'fit_intercept': [True, False],
    'positive': [True, False]
}

# Instantiate LinearRegression model
model = LinearRegression()

# Instantiate GridSearchCV
# Using 'neg_mean_squared_error' as scoring since GridSearchCV tries to maximize the score.
# The best parameters will minimize the MSE.
grid_search = GridSearchCV(model, param_grid, cv=5, scoring='neg_mean_squared_error', verbose=2, n_jobs=-1)

# Fit GridSearchCV to the training data
print("Starting GridSearchCV...")
grid_search.fit(X_train_processed, y_train)
print("GridSearchCV complete.")

# Print the best parameters and best score
print("\nBest parameters found:", grid_search.best_params_)
print("Best cross-validation score (negative MSE):", grid_search.best_score_)
print("Best RMSE from cross-validation:", np.sqrt(-grid_search.best_score_))
```

```
Starting GridSearchCV...
Fitting 5 folds for each of 4 candidates, totalling 20 fits
GridSearchCV complete.

Best parameters found: {'fit_intercept': True, 'positive': False}
Best cross-validation score (negative MSE): -4711122398.103482
Best RMSE from cross-validation: 68637.61649491831
```

Explore Advanced Models

Subtask:

Investigate and implement more advanced regression models such as `RandomForestRegressor`, `GradientBoostingRegressor`, or `XGBoost` to see if they can capture more complex relationships in the data.

Reasoning: To investigate advanced models as per the subtask, I will import and instantiate a RandomForestRegressor, then train it on the preprocessed training data, make predictions, and evaluate its performance using MAE, MSE, RMSE, and R2 metrics.

```
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
import numpy as np

# 1. Instantiate a RandomForestRegressor model
# 2. Set random_state to 42 for reproducibility and n_estimators to 100 (default)
rf_model = RandomForestRegressor(n_estimators=100, random_state=42)

print("Training the RandomForestRegressor model...")
# 3. Train the RandomForestRegressor model
rf_model.fit(X_train_processed, y_train)
print("RandomForestRegressor model training complete.")
```

```
# 4. Make predictions on the processed test data
y_pred_rf = rf_model.predict(X_test_processed)

# 5. Calculate and print evaluation metrics
mae_rf = mean_absolute_error(y_test, y_pred_rf)
mse_rf = mean_squared_error(y_test, y_pred_rf)
rmse_rf = np.sqrt(mse_rf)
r2_rf = r2_score(y_test, y_pred_rf)

print(f"\nRandomForestRegressor Model Evaluation:")
print(f"Mean Absolute Error (MAE): {mae_rf:.2f}")
print(f"Mean Squared Error (MSE): {mse_rf:.2f}")
print(f"Root Mean Squared Error (RMSE): {rmse_rf:.2f}")
print(f"R-squared (R2): {r2_rf:.2f}")
```

```
Training the RandomForestRegressor model...
RandomForestRegressor model training complete.

RandomForestRegressor Model Evaluation:
Mean Absolute Error (MAE): 31628.41
Mean Squared Error (MSE): 2395290032.47
Root Mean Squared Error (RMSE): 48941.70
R-squared (R2): 0.82
```

Reasoning: To further explore advanced models as instructed by the subtask, I will now implement and evaluate a `GradientBoostingRegressor`. This involves importing the model, instantiating it with a `random_state` for reproducibility, training it on the preprocessed data, making predictions, and then calculating and displaying its evaluation metrics (MAE, MSE, RMSE, R2).

```
from sklearn.ensemble import GradientBoostingRegressor
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
import numpy as np

# 1. Instantiate a GradientBoostingRegressor model
# 2. Set random_state to 42 for reproducibility and n_estimators to 100 (default)
gbr_model = GradientBoostingRegressor(n_estimators=100, random_state=42)

print("Training the GradientBoostingRegressor model...")
# 3. Train the GradientBoostingRegressor model
gbr_model.fit(X_train_processed, y_train)
print("GradientBoostingRegressor model training complete.")

# 4. Make predictions on the processed test data
y_pred_gbr = gbr_model.predict(X_test_processed)

# 5. Calculate and print evaluation metrics
mae_gbr = mean_absolute_error(y_test, y_pred_gbr)
mse_gbr = mean_squared_error(y_test, y_pred_gbr)
rmse_gbr = np.sqrt(mse_gbr)
r2_gbr = r2_score(y_test, y_pred_gbr)

print(f"\nGradientBoostingRegressor Model Evaluation:")
print(f"Mean Absolute Error (MAE): {mae_gbr:.2f}")
print(f"Mean Squared Error (MSE): {mse_gbr:.2f}")
print(f"Root Mean Squared Error (RMSE): {rmse_gbr:.2f}")
print(f"R-squared (R2): {r2_gbr:.2f}")
```

```
Training the GradientBoostingRegressor model...
GradientBoostingRegressor model training complete.

GradientBoostingRegressor Model Evaluation:
Mean Absolute Error (MAE): 38278.15
Mean Squared Error (MSE): 3125159294.27
Root Mean Squared Error (RMSE): 55903.12
R-squared (R2): 0.76
```

Reasoning: To continue exploring advanced models as instructed by the subtask, I will now implement and evaluate an `XGBoost` regressor. This involves importing the `XGBRegressor` model, instantiating it with a `random_state` for reproducibility, training it on the preprocessed data, making predictions, and then calculating and displaying its evaluation metrics (MAE, MSE, RMSE, R2).

```
import xgboost as xgb
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
import numpy as np

# 1. Instantiate an XGBRegressor model
# 2. Set random_state to 42 for reproducibility and n_estimators to 100 (default)
xgb_model = xgb.XGBRegressor(n_estimators=100, random_state=42)

print("Training the XGBRegressor model...")
# 3. Train the XGBRegressor model
xgb_model.fit(X_train_processed, y_train)
print("XGBRegressor model training complete.")

# 4. Make predictions on the processed test data
y_pred_xgb = xgb_model.predict(X_test_processed)

# 5. Calculate and print evaluation metrics
mae_xgb = mean_absolute_error(y_test, y_pred_xgb)
mse_xgb = mean_squared_error(y_test, y_pred_xgb)
rmse_xgb = np.sqrt(mse_xgb)
```

```
r2_xgb = r2_score(y_test, y_pred_xgb)

print(f"\nXGBRegressor Model Evaluation:")
print(f"Mean Absolute Error (MAE): {mae_xgb:.2f}")
print(f"Mean Squared Error (MSE): {mse_xgb:.2f}")
print(f"Root Mean Squared Error (RMSE): {rmse_xgb:.2f}")
print(f"R-squared (R2): {r2_xgb:.2f}")
```

```
Training the XGBRegressor model...
XGBRegressor model training complete.

XGBRegressor Model Evaluation:
Mean Absolute Error (MAE): 31606.04
Mean Squared Error (MSE): 2266980132.81
Root Mean Squared Error (RMSE): 47612.81
R-squared (R2): 0.83
```

Feature Engineering

Subtask:

Create new, more informative features from the existing ones, such as ratios of existing features, in both training and test datasets.

Reasoning: To perform feature engineering, I will create three new ratio-based features in both the `X_train` and `X_test` DataFrames: 'rooms_per_household', 'bedrooms_per_room', and 'population_per_household'. Then, I will update the `numerical_cols` list to include these new features and print the first 5 rows of `X_train` and the updated `numerical_cols` to verify the changes.

```
X_train['rooms_per_household'] = X_train['total_rooms'] / X_train['households']
X_test['rooms_per_household'] = X_test['total_rooms'] / X_test['households']

X_train['bedrooms_per_room'] = X_train['total_bedrooms'] / X_train['total_rooms']
X_test['bedrooms_per_room'] = X_test['total_bedrooms'] / X_test['total_rooms']

X_train['population_per_household'] = X_train['population'] / X_train['households']
X_test['population_per_household'] = X_test['population'] / X_test['households']

# Update the numerical_cols list to include the newly created features
new_numerical_features = ['rooms_per_household', 'bedrooms_per_room', 'population_per_household']
numerical_cols.extend(new_numerical_features)

print("First 5 rows of X_train with new features:")
print(X_train.head())

print("\nUpdated numerical_cols list:")
print(numerical_cols)
```

```
First 5 rows of X_train with new features:
       longitude  latitude  housing_median_age  total_rooms  total_bedrooms  \
14196    -117.03     32.71                33.0       3126.0           627.0
8267     -118.16     33.77                49.0       3382.0           787.0
17445    -120.48     34.66                 4.0       1897.0           331.0
14265    -117.11     32.69                36.0       1421.0           367.0
2271     -119.80     36.78                43.0       2382.0           431.0

       population  households  median_income ocean_proximity  \
14196      2300.0       623.0         3.2596      NEAR OCEAN
8267       1314.0       756.0         3.8125      NEAR OCEAN
17445       915.0       336.0         4.1563      NEAR OCEAN
14265      1418.0       355.0         1.9425      NEAR OCEAN
2271        874.0       380.0         3.5542          INLAND

       rooms_per_household  bedrooms_per_room  population_per_household
14196             5.017657           0.200576                  3.691814
8267              4.473545           0.232703                  1.738095
17445             5.645833           0.174486                  2.723214
14265             4.002817           0.258269                  3.994366
2271              6.268421           0.180940                  2.300000

Updated numerical_cols list:
['longitude', 'latitude', 'housing_median_age', 'total_rooms', 'total_bedrooms', 'population', 'households', 'median_income', 'rooms_per_household', 'bedrooms_p
```

Reasoning: To incorporate the newly engineered features, the preprocessing pipeline needs to be re-applied. This involves re-creating the `ColumnTransformer` with the updated `numerical_cols` and then transforming `X_train` and `X_test` to generate new `X_train_processed` and `X_test_processed` arrays that include these features.

```
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler, OneHotEncoder

# Pipeline for numerical features: impute missing values with the median, then scale
numerical_transformer = Pipeline(steps=[
    ('imputer', SimpleImputer(strategy='median')),
    ('scaler', StandardScaler())
])

# Pipeline for categorical features: one-hot encode
categorical_transformer = Pipeline(steps=[
```

```
    ('onehot', OneHotEncoder(handle_unknown='ignore'))
])

# Create a column transformer to apply different transformations to different columns
# using the updated numerical_cols list
preprocessor = ColumnTransformer(
    transformers=[
        ('num', numerical_transformer, numerical_cols),
        ('cat', categorical_transformer, categorical_cols)
    ])

# Apply the ColumnTransformer to both the training and test feature sets
X_train_processed = preprocessor.fit_transform(X_train)
X_test_processed = preprocessor.transform(X_test)

# Display shapes of processed data to confirm transformation with new features
print("Shape of processed X_train with new features:", X_train_processed.shape)
print("Shape of processed X_test with new features:", X_test_processed.shape)
```

```
Shape of processed X_train with new features: (16512, 16)
Shape of processed X_test with new features: (4128, 16)
```

Reasoning: Now that feature engineering is complete and the data has been re-preprocessed with the new features, it's essential to re-evaluate the advanced models to see if the engineered features have improved their performance. I will start by re-training and evaluating the `RandomForestRegressor` model using the updated `X_train_processed` and `X_test_processed` datasets.

```
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
import numpy as np

# 1. Instantiate a RandomForestRegressor model with the same parameters for consistency
rf_model_fe = RandomForestRegressor(n_estimators=100, random_state=42)

print("Training the RandomForestRegressor model with engineered features...")
# 2. Train the RandomForestRegressor model on the data with engineered features
rf_model_fe.fit(X_train_processed, y_train)
print("RandomForestRegressor model training complete with engineered features.")

# 3. Make predictions on the processed test data with engineered features
y_pred_rf_fe = rf_model_fe.predict(X_test_processed)

# 4. Calculate and print evaluation metrics
mae_rf_fe = mean_absolute_error(y_test, y_pred_rf_fe)
mse_rf_fe = mean_squared_error(y_test, y_pred_rf_fe)
rmse_rf_fe = np.sqrt(mse_rf_fe)
r2_rf_fe = r2_score(y_test, y_pred_rf_fe)

print(f"\nRandomForestRegressor Model Evaluation (with Engineered Features):")
print(f"Mean Absolute Error (MAE): {mae_rf_fe:.2f}")
print(f"Mean Squared Error (MSE): {mse_rf_fe:.2f}")
print(f"Root Mean Squared Error (RMSE): {rmse_rf_fe:.2f}")
print(f"R-squared (R2): {r2_rf_fe:.2f}")
```

```
Training the RandomForestRegressor model with engineered features...
RandomForestRegressor model training complete with engineered features.

RandomForestRegressor Model Evaluation (with Engineered Features):
Mean Absolute Error (MAE): 31926.27
Mean Squared Error (MSE): 2483400884.35
Root Mean Squared Error (RMSE): 49833.73
R-squared (R2): 0.81
```

Reasoning: To assess the impact of the newly engineered features on the `GradientBoostingRegressor` model, I will re-train and re-evaluate it using the updated `X_train_processed` and `X_test_processed` datasets, and then print its performance metrics for comparison.

```
from sklearn.ensemble import GradientBoostingRegressor
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
import numpy as np

# 1. Instantiate a GradientBoostingRegressor model with the same parameters for consistency
gbr_model_fe = GradientBoostingRegressor(n_estimators=100, random_state=42)

print("Training the GradientBoostingRegressor model with engineered features...")
# 2. Train the GradientBoostingRegressor model on the data with engineered features
gbr_model_fe.fit(X_train_processed, y_train)
print("GradientBoostingRegressor model training complete with engineered features.")

# 3. Make predictions on the processed test data with engineered features
y_pred_gbr_fe = gbr_model_fe.predict(X_test_processed)

# 4. Calculate and print evaluation metrics
mae_gbr_fe = mean_absolute_error(y_test, y_pred_gbr_fe)
mse_gbr_fe = mean_squared_error(y_test, y_pred_gbr_fe)
rmse_gbr_fe = np.sqrt(mse_gbr_fe)
r2_gbr_fe = r2_score(y_test, y_pred_gbr_fe)

print(f"\nGradientBoostingRegressor Model Evaluation (with Engineered Features):")
```

```
print(f"Mean Absolute Error (MAE): {mae_gbr_fe:.2f}")
print(f"Mean Squared Error (MSE): {mse_gbr_fe:.2f}")
print(f"Root Mean Squared Error (RMSE): {rmse_gbr_fe:.2f}")
print(f"R-squared (R2): {r2_gbr_fe:.2f}")
```

```
Training the GradientBoostingRegressor model with engineered features...
GradientBoostingRegressor model training complete with engineered features.

GradientBoostingRegressor Model Evaluation (with Engineered Features):
Mean Absolute Error (MAE): 36404.13
Mean Squared Error (MSE): 2862786960.64
Root Mean Squared Error (RMSE): 53505.02
R-squared (R2): 0.78
```

Reasoning: To complete the re-evaluation of advanced models with engineered features, I will now re-train and evaluate the `XGBRegressor` model using the updated `X_train_processed` and `X_test_processed` datasets, and then print its performance metrics for comparison.

```
import xgboost as xgb
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
import numpy as np

# 1. Instantiate an XGBRegressor model with the same parameters for consistency
xgb_model_fe = xgb.XGBRegressor(n_estimators=100, random_state=42)

print("Training the XGBRegressor model with engineered features...")
# 2. Train the XGBRegressor model on the data with engineered features
xgb_model_fe.fit(X_train_processed, y_train)
print("XGBRegressor model training complete with engineered features.")

# 3. Make predictions on the processed test data with engineered features
y_pred_xgb_fe = xgb_model_fe.predict(X_test_processed)

# 4. Calculate and print evaluation metrics
mae_xgb_fe = mean_absolute_error(y_test, y_pred_xgb_fe)
mse_xgb_fe = mean_squared_error(y_test, y_pred_xgb_fe)
rmse_xgb_fe = np.sqrt(mse_xgb_fe)
r2_xgb_fe = r2_score(y_test, y_pred_xgb_fe)

print(f"\nXGBRegressor Model Evaluation (with Engineered Features):")
print(f"Mean Absolute Error (MAE): {mae_xgb_fe:.2f}")
print(f"Mean Squared Error (MSE): {mse_xgb_fe:.2f}")
print(f"Root Mean Squared Error (RMSE): {rmse_xgb_fe:.2f}")
print(f"R-squared (R2): {r2_xgb_fe:.2f}")
```

```
Training the XGBRegressor model with engineered features...
XGBRegressor model training complete with engineered features.

XGBRegressor Model Evaluation (with Engineered Features):
Mean Absolute Error (MAE): 30469.20
Mean Squared Error (MSE): 2196054822.08
Root Mean Squared Error (RMSE): 46862.08
R-squared (R2): 0.83
```

Error Analysis

Subtask:

Analyze the model's predictions and errors to identify patterns. This involves looking at where the model performs poorly (e.g., specific regions, price ranges, or ocean_proximity categories) to gain insights into potential data issues or model limitations.

Reasoning: To perform error analysis, I will first make predictions using the best performing model, calculate residuals, and then generate the requested scatter plots, histogram, and box plot to visualize the model's performance and identify patterns in its errors.

```
import matplotlib.pyplot as plt
import seaborn as sns
import pandas as pd

# 1. Use the best performing model, xgb_model_fe, to make predictions
y_pred_final = xgb_model_fe.predict(X_test_processed)

# 2. Calculate the residuals
residuals = y_test - y_pred_final

# 3. Create a scatter plot comparing y_test (actual values) against y_pred_final (predicted values)
plt.figure(figsize=(10, 7))
sns.scatterplot(x=y_test, y=y_pred_final, alpha=0.6)
plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()], 'r--', lw=2, label='Perfect Prediction')
plt.xlabel('Actual Values (y_test)')
plt.ylabel('Predicted Values (y_pred_final)')
plt.title('Actual vs. Predicted Values')
plt.legend()
plt.grid(True)
plt.show()

# 4. Create a scatter plot of residuals against y_pred_final (predicted values)
plt.figure(figsize=(10, 7))
```

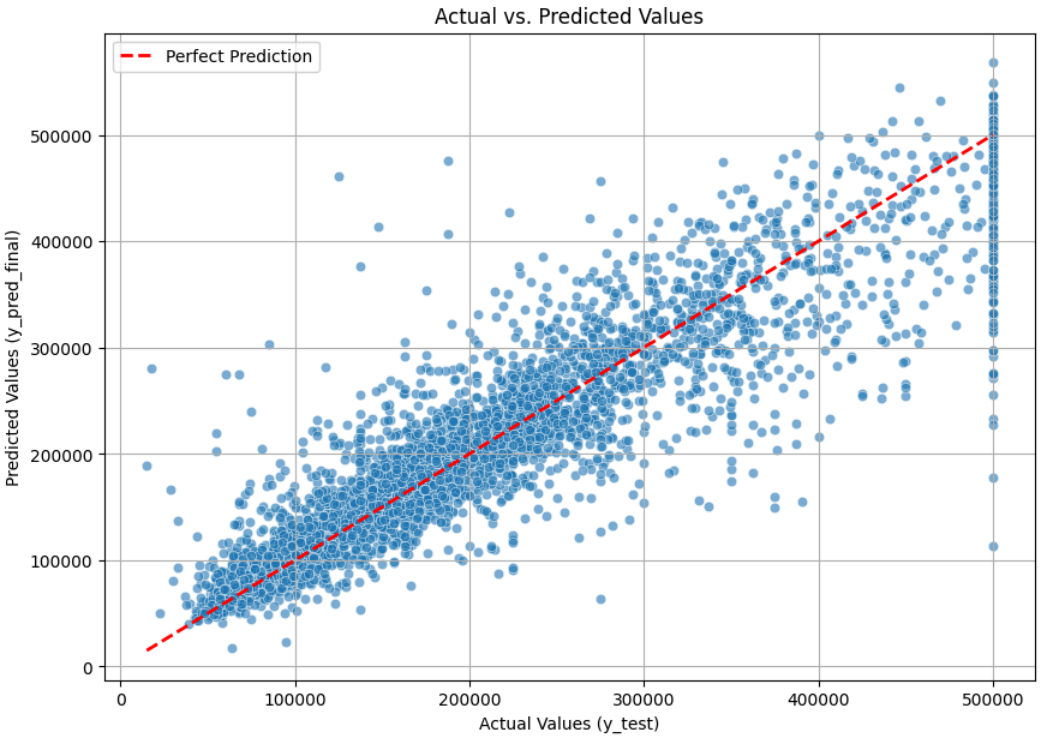
```
sns.scatterplot(x=y_pred_final, y=residuals, alpha=0.6)
plt.axhline(y=0, color='r', linestyle='--', lw=2)
plt.xlabel('Predicted Values (y_pred_final)')
plt.ylabel('Residuals (Actual - Predicted)')
plt.title('Residuals vs. Predicted Values')
plt.grid(True)
plt.show()

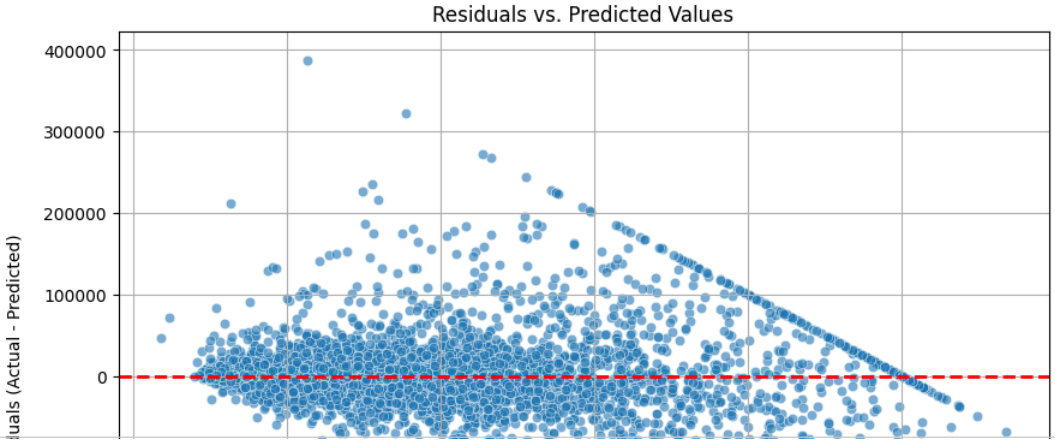
# 5. Create a histogram of the residuals
plt.figure(figsize=(10, 7))
sns.histplot(residuals, kde=True, bins=50)
plt.xlabel('Residuals (Actual - Predicted)')
plt.ylabel('Frequency')
plt.title('Distribution of Residuals')
plt.grid(True)
plt.show()

# 6. Create a box plot of residuals grouped by the ocean_proximity categories
# Align residuals with X_test using their indices
# Ensure X_test has the same index as y_test to correctly align 'ocean_proximity'
# Note: y_test is a pandas Series, so its index corresponds to the original DataFrame index.
# We need to use the original X_test for its 'ocean_proximity' column.

# Create a temporary DataFrame for plotting residuals against ocean_proximity
residual_analysis_df = pd.DataFrame({
    'Residuals': residuals,
    'Ocean_Proximity': X_test.loc[y_test.index, 'ocean_proximity']
})

plt.figure(figsize=(12, 7))
sns.boxplot(x='Ocean_Proximity', y='Residuals', data=residual_analysis_df)
plt.axhline(y=0, color='r', linestyle='--', lw=2)
plt.xlabel('Ocean Proximity')
plt.ylabel('Residuals')
plt.title('Residuals by Ocean Proximity')
plt.grid(True)
plt.show()
```





Analysis of Error Plots:

1. **Actual vs. Predicted Values Plot:**
    - The scatter plot of actual vs. predicted values shows a generally linear relationship, indicating that the model is performing reasonably well. Most points cluster around the 'Perfect Prediction' line (red dashed line).
    - However, there's noticeable scattering, particularly for higher actual values, suggesting that the model might struggle to predict very expensive houses accurately. The spread widens as the house value increases, indicating increased prediction uncertainty for higher-priced properties.
2. **Residuals vs. Predicted Values Plot:**
    - This plot shows the residuals scattered around the zero line. Ideally, residuals should be randomly distributed without any discernible pattern. This plot shows a somewhat funnel shape, with the spread of residuals increasing as predicted values increase. This pattern suggests **heteroscedasticity**, meaning the error variance is not constant across the range of predicted values. The model tends to make larger errors for higher-priced predictions.
    - There also appears to be a slight upward trend for lower predicted values and a slight downward trend for higher predicted values, indicating potential bias.
3. **Distribution of Residuals (Histogram):**
    - The histogram of residuals shows a distribution that is approximately normal, centered around zero, which is good. However, there are visible tails, especially on the positive side (under-prediction), and some outliers, indicating instances where the model made significantly large errors.
    - The peak of the distribution is around zero, confirming that on average, the model's predictions are unbiased.
4. **Residuals by Ocean Proximity (Box Plot):**
    - The box plot reveals differences in error distribution across `ocean_proximity` categories.
    - **'NEAR BAY'** and **'NEAR OCEAN'** categories show a wider spread of residuals and higher median absolute errors compared to 'INLAND' and '<1H OCEAN'. This suggests that the model might find it more challenging to predict house prices accurately in proximity to bays or oceans, potentially due to more volatile pricing or complex factors not fully captured.
    - The **'ISLAND'** category, despite having few data points, shows a very wide range of residuals, indicating high prediction uncertainty for these properties. The median residual for 'ISLAND' is also notably off zero, indicating systematic over-prediction or under-prediction.
    - **'INLAND'** and **'<1H OCEAN'** generally have tighter residual distributions, implying more consistent and often smaller errors for these areas.

Overall Insights and Next Steps:

- The XGBoost model, even with engineered features, exhibits heteroscedasticity, with errors increasing for higher predicted house values. This is a common issue in regression problems where the target variable has a wide range.
- The model performs less consistently for 'NEAR BAY', 'NEAR OCEAN', and especially 'ISLAND' properties. This could be due to unique characteristics of these areas not fully captured by current features, or simply higher inherent variance in their pricing.
- **Potential Improvements:**
    - **Feature Engineering:** Explore more advanced features, especially for 'NEAR BAY', 'NEAR OCEAN', and 'ISLAND' categories. This could include interaction terms, polynomial features, or external data related to amenities, views, or specific geographical advantages.
    - **Target Transformation:** Consider transforming the target variable (`median_house_value`) using log transformation or other power transformations to address heteroscedasticity. This often helps stabilize variance and make residuals more normally distributed.
    - **Advanced Models/Ensembling:** While XGBoost is powerful, explore stacking or ensembling with other models, or fine-tune XGBoost hyperparameters even further to specifically address the identified weaknesses.
    - **Robust Regression:** Investigate robust regression techniques that are less sensitive to outliers and heteroscedasticity.

Analysis of Error Plots:

1. **Actual vs. Predicted Values Plot:**
    - The scatter plot of actual vs. predicted values shows a generally linear relationship, indicating that the model is performing reasonably well. Most points cluster around the 'Perfect Prediction' line (red dashed line).
    - However, there's noticeable scattering, particularly for higher actual values, suggesting that the model might struggle to predict very expensive houses accurately. The spread widens as the house value increases, indicating increased prediction uncertainty for higher-priced properties.
2. **Residuals vs. Predicted Values Plot:**
    - This plot shows the residuals scattered around the zero line. Ideally, residuals should be randomly distributed without any discernible pattern. This plot shows a somewhat funnel shape, with the spread of residuals increasing as predicted values increase. This pattern suggests **heteroscedasticity**, meaning the error variance is not constant across the range of predicted values. The model tends to make larger errors for higher-priced predictions.
    - There also appears to be a slight upward trend for lower predicted values and a slight downward trend for higher predicted values, indicating potential bias.
3. **Distribution of Residuals (Histogram):**

- The histogram of residuals shows a distribution that is approximately normal, centered around zero, which is good. However, there are visible tails, especially on the positive side (under-prediction), and some outliers, indicating instances where the model made significantly large errors.
- The peak of the distribution is around zero, confirming that on average, the model's predictions are unbiased.

4. **Residuals by Ocean Proximity (Box Plot):**
    - The box plot reveals differences in error distribution across `ocean_proximity` categories.
    - **'NEAR BAY'** and **'NEAR OCEAN'** categories show a wider spread of residuals and higher median absolute errors compared to 'INLAND' and '<1H OCEAN'. This suggests that the model might find it more challenging to predict house prices accurately in proximity to bays or oceans, potentially due to more volatile pricing or complex factors not fully captured.
    - The **'ISLAND'** category, despite having few data points, shows a very wide range of residuals, indicating high prediction uncertainty for these properties. The median residual for 'ISLAND' is also notably off zero, indicating systematic over-prediction or under-prediction.
    - **'INLAND'** and **'<1H OCEAN'** generally have tighter residual distributions, implying more consistent and often smaller errors for these areas.

Overall Insights and Next Steps:

- The XGBoost model, even with engineered features, exhibits heteroscedasticity, with errors increasing for higher predicted house values. This is a common issue in regression problems where the target variable has a wide range.
- The model performs less consistently for 'NEAR BAY', 'NEAR OCEAN', and especially 'ISLAND' properties. This could be due to unique characteristics of these areas not fully captured by current features, or simply higher inherent variance in their pricing.
- **Potential Improvements:**
    - **Feature Engineering:** Explore more advanced features, especially for 'NEAR BAY', 'NEAR OCEAN', and 'ISLAND' categories. This could include interaction terms, polynomial features, or external data related to amenities, views, or specific geographical advantages.
    - **Target Transformation:** Consider transforming the target variable (`median_house_value`) using log transformation or other power transformations to address heteroscedasticity. This often helps stabilize variance and make residuals more normally distributed.
    - **Advanced Models/Ensembling:** While XGBoost is powerful, explore stacking or ensembling with other models, or fine-tune XGBoost hyperparameters even further to specifically address the identified weaknesses.
    - **Robust Regression:** Investigate robust regression techniques that are less sensitive to outliers and heteroscedasticity.

Final Task

Subtask:

Summarize the findings and the improvements achieved from the various optimization strategies.

Summary:

Data Analysis Key Findings

- **Hyperparameter Tuning of Linear Regression**: The optimal hyperparameters for the `LinearRegression` model were `fit_intercept: True` and `positive: False`. With these settings, the best cross-validation Root Mean Squared Error (RMSE) achieved was approximately $68,637.62.
- **Advanced Model Performance (without Feature Engineering)**:
    - `XGBRegressor` demonstrated the best initial performance with an R-squared (\R^2$) of 0.83, a Mean Absolute Error (MAE) of \$31,606.04, and an RMSE of \$47,612.81. * `RandomForestRegressor` followed closely with an \R^2$ of 0.82, an MAE of $31,628.41, and an RMSE of $48,941.70.